

BOUCHUT Tanguy
Grenoble INP - PHELMMA

RAPPORT ARCHITECTURE AVANCÉE

Conception d'un cœur RISCv

Table des matières

1	Conception de l'architecture matérielle	2
1.1	FETCH	2
1.1.1	Le PC FETCH	2
1.1.2	Le testbench du PC FETCH	2
1.1.3	Program ROM	3
1.1.4	Le testbench du program ROM	3
1.2	DECODE	4
1.2.1	Instruction DECODER	4
1.2.2	Immediate DECODER	5
1.2.3	Register file	6
1.3	EXECUTE	7
1.3.1	Store unit	7
1.3.2	ALU	8
1.4	MEMORY	9
1.4.1	Data RAM	9
1.4.2	Le testbench du data RAM	10
1.4.3	Load unit	10
2	Simulation	12
2.1	RISCV	12
2.2	RISCV_5STG pipeline	16

1 Conception de l'architecture matérielle

Tout d'abord, il a fallu implémenter tous les composants qui font partis du cœur RISC-V. Nous les avons implémentés dans l'ordre dans lequel ils interviennent pendant un cycle d'horloge.

1.1 FETCH

1.1.1 Le PC FETCH

Tout d'abord nous avons commencés par le PC fetch qui détermine et fournit l'adresse de la prochaine instruction à exécuter.

```
24 process (CLK)
25 begin
26     if rising_edge(clk) then
27         -- Soit un reset classique
28         if resetn = '0' then
29             pc_addr <= (others => '0');
30
31         -- incrément classique du PC
32         elsif enable_f = '1' then
33             pc_addr <= pc_addr + to_unsigned(4, 32);
34
35         -- jump ou branchement
36         elsif enable_m = '1' and jumporbranch = '1' then
37             pc_addr <= unsigned (jumporbranchaddress);
38         end if;
39     end if;
40 end process;
41
42 -- On lie le signal interne pc_addr à la sortie pc_value
43 pc_value <= std_logic_vector(pc_addr);
```

1.1.2 Le testbench du PC FETCH

```
47 stimulus: process
48 begin
49     resetn <= '0';
50     enable_f <= '0';
51     enable_m <= '0';
52     jumpOrBranch <= '0';
53     jumpOrBranchAddress <= (others => '0');
54     wait for clock_period;
55     assert pc_value = x"00000000" report "pc_value has to be reset" severity error;
56
57     resetn <= '1';
58     enable_f <= '1';
59     wait for clock_period;
60     assert pc_value = x"00000004" report "pc_value has to be 4" severity error;
61
62     enable_f <= '0';
63     enable_m <= '1';
64     wait for clock_period;
65     assert pc_value = x"00000004" report "pc_value doesn't have to change" severity error;
66
67     enable_m <= '0';
68     jumpOrBranch <= '1';
69     wait for clock_period;
70     assert pc_value = x"00000004" report "pc_value doesn't have to change" severity error;
71
72     enable_m <= '1';
73     wait for clock_period;
```

```

74     assert pc_value = x"00000000" report "pc_value has to jump to 0 (jumpOrBranchAddress" severity
       ↪ error;
75
76     stop_the_clock <= true;
77     wait;
78 end process;

```

1.1.3 Program ROM

Ensuite, nous nous sommes concentrés sur le program ROM qui lit la mémoire et renvoie l'instruction correspondant à l'adresse demandée.

```

48     ADDR <= ADDR_R(ROM_ADDR-1 downto 2);
49
50     process(CLOCK)
51     begin
52         if rising_edge(CLOCK) then
53             if ENABLE = '1' then
54                 DATA_0 <= memory(to_integer(unsigned(ADDR)));
55             end if;
56         end if;
57     end process;

```

1.1.4 Le testbench du program ROM

Pour écrire le testbench du ROM, il a fallu faire plusieurs modifications au fichier original. Tout d'abord, il a fallu importer les fichiers de configuration.

```

9     use work.riscv_types.all;
10    use work.riscv_config.all;

```

Ensuite, il a fallu corriger les erreurs en remplaçant RAM_ADDR_BITS par ROM_ADDR.

```

23     ADDR_R : IN STD_LOGIC_VECTOR(ROM_ADDR-1 DOWNT0 0);

```

```

30    signal ADDR_R: STD_LOGIC_VECTOR(ROM_ADDR-1 DOWNT0 0);

```

Corriger le generic map.

```

39    uut: mem_rom    port map ( CLOCK      => CLOCK,
40                               ENABLE      => ENABLE,
41                               ADDR_R      => ADDR_R,
42                               DATA_0     => DATA_0 );

```

Et enfin écrire le testbench.

```

44    stimulus: process
45    begin
46
47        ENABLE <= '0';
48        ADDR_R <= (others => '0');
49        wait for clock_period;
50
51        -- =====
52        -- ATTENTION : En fonction de firmware/PROGROM.mem, l'instruction 1 est différente ! (à faire
       ↪ concorder les 2 pour avoir un bon test).
53        -- =====
54

```

```

55  -- =====
56  -- TEST 1 : On vérifie qu'à l'adresse 4, on ressort bien l'instruction 1.
57  -- =====
58  ENABLE <= '1';
59  ADDR_R <= std_logic_vector(to_unsigned(4, ROM_ADDR));
60  wait for clock_period;
61  assert DATA_0 = x"04800713" report "DATA_0 ne vaut pas l'instruction 1 : 04800713" severity
62  ↪ error;
63
64  -- =====
65  -- TEST 2 : On vérifie qu'à ENABLE='0', DATA_0 n'est pas modifié.
66  -- =====
67  ENABLE <= '0';
68  ADDR_R <= std_logic_vector(to_unsigned(0, ROM_ADDR));
69  wait for clock_period;
70  assert DATA_0 = x"04800713" report "DATA_0 ne vaut pas l'instruction 1 : 04800713 (il a été
71  ↪ modifié malgré ENABLE=0)" severity error;
72
73  stop_the_clock <= true;
74  wait;
75  end process;

```

1.2 DECODE

1.2.1 Instruction DECODER

Le décodeur d'instruction analyse l'instruction pour comprendre quelle opération doit être réalisée.

```

42  SIGNAL opcode : STD_LOGIC_VECTOR(6 downto 0);
43  SIGNAL imm12sys : STD_LOGIC_VECTOR(31 downto 20);
44
45  begin
46
47  -- On extrait les champs qui sont presque toujours à la même place en RISC-V
48  opcode <= instr_i(6 downto 0);
49  imm12sys <= instr_i(31 downto 20);
50  funct3_o <= instr_i(14 downto 12);
51  funct7_o <= instr_i(31 downto 25);
52  rs1_o <= instr_i(19 downto 15);
53  rs2_o <= instr_i(24 downto 20);
54  rdId_o <= instr_i(11 downto 7);
55
56
57  -- To be completed
58  process (opcode, funct3_o, imm12sys)
59  begin
60
61      isLoad_o <= '0';
62      isStore_o <= '0';
63      isALUreg_o <= '0';
64      isBranch_o <= '0';
65      isSYSTEM_o <= '0';
66      isJAL_o <= '0';
67      isJALR_o <= '0';
68      isJALorJALR_o <= '0';
69      isAuipc_o <= '0';
70      isLui_o <= '0';
71      isCustom_o <= '0';
72      isCSRrs_o <= '0';
73      isEBreak_o <= '0';
74      isByte_o <= '0';
75      isHalf_o <= '0';
76
77  case opcode is

```

```

78     when "0000011" =>
79         isLoad_o <= '1';
80         if (funct3_o = "000") OR (funct3_o = "100") then
81             isByte_o <= '1';
82         elsif (funct3_o = "001") OR (funct3_o = "101") then
83             isHalf_o <= '1';
84         end if;
85
86     when "0100011" =>
87         isStore_o <= '1';
88         if (funct3_o = "000") then
89             isByte_o <= '1';
90         elsif (funct3_o = "001") then
91             isHalf_o <= '1';
92         end if;
93
94     when "0110011" =>
95         isALUreg_o <= '1';
96
97     when "0010011" =>
98         null;
99
100    when "1100011" =>
101        isBranch_o <= '1';
102
103    when "1110011" =>
104        isSYSTEM_o <= '1';
105        if (imm12sys = x"001") or (imm12sys = x"000") then --RAJOUT de ECALL instruction
106            ↪ sinon le test ne passait pas ligne 2710
107            isEBreak_o <= '1';
108        end if;
109
110    when "1101111" =>
111        isJAL_o <= '1';
112        isJALorJALR_o <= '1';
113
114    when "1100111" =>
115        isJALR_o <= '1';
116        isJALorJALR_o <= '1';
117
118    when "0010111" =>
119        isAuipc_o <= '1';
120
121    when "0110111" =>
122        isLui_o <= '1';
123
124    when others =>
125        isCustom_o <= '1';
126
127    end case;
128
129    end process;

```

1.2.2 Immediate DECODER

Le décodeur d'immédiat extrait et formate les valeurs constantes (immédiates) intégrées directement dans l'instruction.

```

47    Iimm <= repeat_bit(INSTR(31), 20) & INSTR(31 downto 20);
48    Simm <= repeat_bit(INSTR(31), 20) & INSTR(31 downto 25) & INSTR(11 downto 7);
49    Uimm <= INSTR(31 downto 12) & x"000";
50    Jimm <= repeat_bit(INSTR(31), 12) & INSTR(19 downto 12) & INSTR(20) & INSTR(30 downto 21) & '0';
51    Bimm <= repeat_bit(INSTR(31), 20) & INSTR(7) & INSTR(30 downto 25) & INSTR(11 downto 8) & '0';
52

```

```

53 process(all)
54 begin
55     if isStore = '1' then
56         imm <= Simm;
57     elsif isbranch = '1' then
58         imm <= Bimm;
59     elsif isJAL = '1' then
60         imm <= Jimm;
61     elsif (isAuipc = '1') or (isLui = '1') then
62         imm <= Uimm;
63     else
64         imm <= Iimm;
65     end if;
66 end process;

```

1.2.3 Register file

Le fichier de registre lit et fournit les valeurs des opérandes stockées dans les registres du processeur.

```

41 DATA_rs1 <= registerFile(to_integer(unsigned(RS1_id)));
42 DATA_rs2 <= registerFile(to_integer(unsigned(RS2_id)));
43
44 process(CLOCK)
45 begin
46     if rising_edge(CLOCK) then
47         if RD_id_we = '1' then
48             if RD_id /= "00000" then
49                 registerFile(to_integer(unsigned(RD_id))) <= DATA_rd;
50             end if;
51         end if;
52     end if;
53 end process;

```

```

54 RS1_id <= "00000";
55 RS2_id <= "00000";
56 RD_id <= "00000";
57 RD_id_we <= '0';
58 DATA_rd <= (others => '0');
59 wait for clock_period;
60
61 -- =====
62 -- TEST 1 : Écriture dans un registre normal (ex: x5) et lecture en RS1.
63 -- =====
64
65 RD_id <= "00101"; --5
66 DATA_rd <= x"FFFFFFFF";
67 RD_id_we <= '1';
68 wait for clock_period;
69
70 RD_id_we <= '0';
71 RS1_id <= "00101";
72 wait for clock_period;
73 assert DATA_rs1 = x"FFFFFFFF" report "Erreur: L'écriture dans x5 a échoué" severity error;
74
75 -- =====
76 -- TEST 2 : Lecture en RS2.
77 -- =====
78
79 RS2_id <= "00101";
80 wait for clock_period;

```

```

81  assert DATA_rs2 = x"FFFFFFFF" report "Erreur: La lecture en RS2 a échoué" severity error;
82
83  -- =====
84  -- TEST 3 : Écriture dans le registre zéro x0 et lecture en RS1.
85  -- =====
86
87  RD_id <= "00000"; --0
88  DATA_rd <= x"FFFFFFFF";
89  RD_id_we <= '1';
90  wait for clock_period;
91
92  RD_id_we <= '0';
93  RS1_id <= "00000";
94  wait for clock_period;
95  assert DATA_rs1 = x"00000000" report "Erreur: L'écriture dans x0 a fonctionné alors qu'elle
96  ↪  aurait dû échouer" severity error;
97
98  -- =====
99  -- TEST 4 : Écriture sans activation.
100  -- =====
101
102  RD_id_we <= '0';
103  RD_id <= "00101"; --5
104  DATA_rd <= x"00000000";
105  wait for clock_period;
106
107  RS1_id <= "00101";
108  wait for clock_period;
109  assert DATA_rs1 = x"FFFFFFFF" report "Erreur: L'écriture dans x5 a fonctionné alors qu'RD_id_we =
110  ↪  0" severity error;
111
112  stop_the_clock <= true;
113  wait;
114  end process;

```

Le testbench du register file

1.3 EXECUTE

1.3.1 Store unit

L'unité de stockage prépare l'adresse et les données en vue d'une écriture en mémoire.

```

23  process(ADDR_W, DATA_W, is_byte, is_half)
24  begin
25      data_mask <= "1111";
26      data_value <= DATA_W;
27
28      if is_byte = '1' then
29          case ADDR_W is
30              when "00" =>
31                  data_mask <= "0001";
32                  data_value <= x"000000" & DATA_W(7 downto 0);
33              when "01" =>
34                  data_mask <= "0010";
35                  data_value <= x"0000" & DATA_W(7 downto 0) & x"00";
36              when "10" =>
37                  data_mask <= "0100";
38                  data_value <= x"00" & DATA_W(7 downto 0) & x"0000";
39              when "11" =>
40                  data_mask <= "1000";
41                  data_value <= DATA_W(7 downto 0) & x"000000";
42              when others =>
43                  data_mask <= "0000";
44                  data_value <= (others => '0');

```

```

45     end case;
46
47     elsif is_half = '1' then
48         if ADDR_W(1) = '0' then
49             data_mask <= "0011";
50             data_value <= x"0000" & DATA_W(15 downto 0);
51         else
52             data_mask <= "1100";
53             data_value <= DATA_W(15 downto 0) & x"0000";
54         end if;
55     end if;
56 end process;

```

1.3.2 ALU

L'unité arithmétique et logique effectue les calculs mathématiques et les opérations logiques demandés.

```

48 signal operand1 : STD_LOGIC_VECTOR(31 downto 0);
49 signal operand2 : STD_LOGIC_VECTOR(31 downto 0);
50
51 begin
52
53     operand1 <= rs1_v;
54     operand2 <= rs2_v when (isALUreg = '1' or isBranch = '1') else imm_v;
55
56     aluPlus_v <= std_logic_vector(unsigned(operand1) + unsigned(operand2));
57
58     process(operand1, operand2, isAluSubstraction, isBranch, isCustom, func3, aluPlus_v)
59     begin
60         aluOut_v <= x"00000000";
61         takeBranch <= '0';
62         if isBranch = '1' then
63             case func3 is
64                 when "000" => -- BEQ
65                     if unsigned(operand1) = unsigned(operand2) then
66                         takeBranch <= '1';
67                     end if;
68                 when "001" => -- BNE
69                     if unsigned(operand1) /= unsigned(operand2) then
70                         takeBranch <= '1';
71                     end if;
72                 when "101" => -- BGE
73                     if signed(operand1) >= signed(operand2) then
74                         takeBranch <= '1';
75                     end if;
76                 when "111" => -- BGEU
77                     if unsigned(operand1) >= unsigned(operand2) then
78                         takeBranch <= '1';
79                     end if;
80                 when "110" => -- BLTU
81                     if unsigned(operand1) < unsigned(operand2) then
82                         takeBranch <= '1';
83                     end if;
84                 when "100" => -- BLT
85                     if signed(operand1) < signed(operand2) then
86                         takeBranch <= '1';
87                     end if;
88                 when others =>
89                     null;
90             end case;
91         else
92             case func3 is
93                 when "000" => -- ADD or SUB

```

```

94         if isAluSubstraction = '1' then
95             aluOut_v <= std_logic_vector(unsigned(operande1) - unsigned(operande2));
96         else
97             aluOut_v <= aluPlus_v;
98         end if;
99     when "001" => -- SLL(i)
100         aluOut_v <= std_logic_vector(shift_left(unsigned(operande1),
101             ↪ to_integer(unsigned(operande2(4 downto 0)))));
102     when "011" => -- SLT(i)U
103         if unsigned(operande1) < unsigned(operande2) then
104             aluOut_v <= x"00000001";
105         else
106             aluOut_v <= x"00000000";
107         end if;
108     when "010" => -- SLT(i)
109         if signed(operande1) < signed(operande2) then
110             aluOut_v <= x"00000001";
111         else
112             aluOut_v <= x"00000000";
113         end if;
114     when "110" => -- OR(i)
115         aluOut_v <= operande1 OR operande2;
116     when "100" => -- XOR(i)
117         aluOut_v <= operande1 XOR operande2;
118     when "101" => -- SRL(i) or SRA(i)
119         if isAluSubstraction = '1' then
120             aluOut_v <= std_logic_vector(shift_right(signed(operande1),
121             ↪ to_integer(unsigned(operande2(4 downto 0)))));
122         else
123             aluOut_v <= std_logic_vector(shift_right(unsigned(operande1),
124             ↪ to_integer(unsigned(operande2(4 downto 0)))));
125         end if;
126     when "111" => -- AND(i)
127         aluOut_v <= operande1 AND operande2;
128     when others =>
129         null;
130 end case;
131 end if;
132 end process;

```

1.4 MEMORY

1.4.1 Data RAM

La mémoire de données exécute concrètement les opérations de lecture ou d'écriture dans la mémoire de données.

```

49     R_ADDR <= ADDR_RW(RAM_ADDR-1 downto 2);
50     DATA_R <= memory(to_integer(unsigned(R_ADDR)));
51
52     process(CLOCK)
53     begin
54         if rising_edge(CLOCK) then
55             if ENABLE = '1' then
56                 if WRITE_M(3) = '1' then
57                     memory(to_integer(unsigned(R_ADDR)))(RAM_WIDTH-1 downto 24) <= DATA_W(RAM_WIDTH-1
58                     ↪ downto 24);
59                 end if;
60                 if WRITE_M(2) = '1' then
61                     memory(to_integer(unsigned(R_ADDR)))(24-1 downto 16) <= DATA_W(24-1 downto 16);
62                 end if;
63                 if WRITE_M(1) = '1' then
64                     memory(to_integer(unsigned(R_ADDR)))(16-1 downto 8) <= DATA_W(16-1 downto 8);
65                 end if;

```

```

65         if WRITE_M(0) = '1' then
66             memory(to_integer(unsigned(R_ADDR))(8-1 downto 0) <= DATA_W(8-1 downto 0);
67         end if;
68     end if;
69 end if;
70 end process;

```

1.4.2 Le testbench du data RAM

Pour le testbench du data RAM, il faut faire les mêmes modifications qui ont été faites pour le program ROM que nous n'allons pas détailler de nouveau ici.

```

61 -- =====
62 -- ATTENTION : En fonction de firmware/DATARAM.mem, la donnée est différente ! (à faire concorder
63 -- les 2 pour avoir un bon test).
64 -- =====
65
66 -- =====
67 -- TEST 1 : On vérifie qu'à l'adresse 4, on ressort bien la donnée 1.
68 -- =====
69 ENABLE <= '1';
70 ADDR_RW <= std_logic_vector(to_unsigned(4, RAM_ADDR));
71 wait for clock_period;
72 assert DATA_R = x"00000000" report "DATA_R ne vaut pas l'instruction 1 : 00020137" severity
73     error;
74
75 -- =====
76 -- TEST 2 : On vérifie qu'à ENABLE='0', DATA_0 n'est pas modifié.
77 -- =====
78 ENABLE <= '0';
79 ADDR_RW <= std_logic_vector(to_unsigned(0, RAM_ADDR));
80 wait for clock_period;
81 assert DATA_R = x"00000000" report "DATA_R ne vaut pas la donnée 1 : 00020137 (il a été modifié
82     malgré ENABLE=0)" severity error;

```

1.4.3 Load unit

L'unité de chargement formate et adapte les données qui viennent d'être extraites de la mémoire.

```

25 with ADDR_R select
26     M_LOAD_B <= DATA_R( 7 downto 0) when "00",
27                DATA_R(15 downto 8) when "01",
28                DATA_R(23 downto 16) when "10",
29                DATA_R(31 downto 24) when others;
30
31 with ADDR_R(1) select
32     M_LOAD_H <= DATA_R(31 downto 16) when '1',
33                DATA_R(15 downto 0) when others;
34
35 process(DATA_R, M_LOAD_B, M_LOAD_H, is_byte, is_half, is_sign_ext)
36 begin
37     if is_byte = '1' then
38         if is_sign_ext = '1' and M_LOAD_B(7) = '1' then
39             data_value <= x"FFFFFF" & M_LOAD_B;
40         else
41             data_value <= x"000000" & M_LOAD_B;
42         end if;
43     elsif is_half = '1' then
44         if is_sign_ext = '1' and M_LOAD_H(15) = '1' then
45             data_value <= x"FFFF" & M_LOAD_H;
46         else
47             data_value <= x"0000" & M_LOAD_H;

```

```
48     end if;  
49     else  
50         data_value <= DATA_R;  
51     end if;  
52 end process;
```

2 Simulation

Une fois tous les composants du cœur vérifiés (make *composant*), nous passons à la simulation du cœur complet RISC-V et RISC-V_5STG sur les différents tests créés dans `soft/apps/`

2.1 RISC-V

Notre cœur RISC-V non pipeliné fonctionne en simulation sur la majorité des tests : 1-hello_putc, 2-hello_putchar, 3-hello_print, 3-vector-add, 4-hello_printf, 5-riscv_logo_uart et 7-dhrystone.

Notre cœur RISC-V non pipeliné ne fonctionne par contre pas sur le test 6-riscv_logo_oled ce qui est normal puisque ce test fonctionne seulement sur FPGA.

Les autres tests n'ont pas été traités.

A terminal window with a dark purple background and light blue text. The text displayed is "Hello !" on the first line and "simulation finished @875ns" on the second line.

FIGURE 1 – Résultat du test 1-hello_putc

A terminal window with a dark purple background and light blue text. The text displayed is "Hello" on the first line and "simulation finished @975ns" on the second line.

FIGURE 2 – Résultat du test 2-hello_putchar

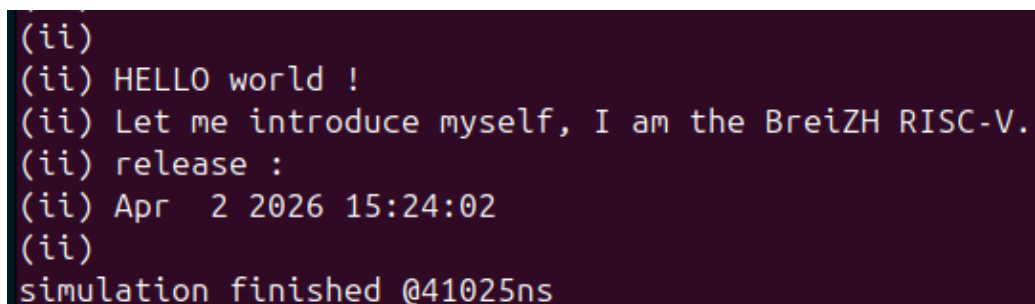
A terminal window with a dark purple background and light blue text. The text displayed is a multi-line output: "(ii)", "(ii) HELLO world !", "(ii) Let me introduce myself, I am the Breizh RISC-V.", "(ii) release :", "(ii) Apr 2 2026 15:24:02", "(ii)", and "simulation finished @41025ns".

FIGURE 3 – Résultat du test 3-hello_print


```

Dhrystone Benchmark, Version 2.1 (Language: C)
Program compiled without 'register' attribute
Please give the number of runs through the benchmark:
Execution starts, 100 runs through Dhrystone
Execution ends
Final values of the variables used in the benchmark:
Int_Glob:          5
    should be:    5
Bool_Glob:         1
    should be:    1
Ch_1_Glob:         A
    should be:    A
Ch_2_Glob:         B
    should be:    B
Arr_1_Glob[8]:     7
    should be:    7
Arr_2_Glob[8][7]:  110
    should be:    Number_Of_Runs + 10
Ptr_Glob->
  Ptr_Comp:        130912
    should be:    (implementation-dependent)
  Discr:           0
    should be:    0
  Enum_Comp:       2
    should be:    2
  Int_Comp:        17
    should be:    17
  Str_Comp:        DHRYSTONE PROGRAM, SOME STRING
    should be:    DHRYSTONE PROGRAM, SOME STRING
Next_Ptr_Glob->
  Ptr_Comp:        130912
    should be:    (implementation-dependent), same as above

```

FIGURE 7 – Résultat du test 7-dhrystone – Partie 1

```

Discr: 0
    should be: 0
Enum_Comp: 1
    should be: 1
Int_Comp: 18
    should be: 18
Str_Comp: DHRYSTONE PROGRAM, SOME STRING
    should be: DHRYSTONE PROGRAM, SOME STRING
Int_1_Loc: 5
    should be: 5
Int_2_Loc: 13
    should be: 13
Int_3_Loc: 7
    should be: 7
Enum_Loc: 1
    should be: 1
Str_1_Loc: DHRYSTONE PROGRAM, 1'ST STRING
    should be: DHRYSTONE PROGRAM, 1'ST STRING
Str_2_Loc: DHRYSTONE PROGRAM, 2'ND STRING
    should be: DHRYSTONE PROGRAM, 2'ND STRING
Number_Of_Runs: 100
User_Time: 0 cycles, 0 insn
Cycles_Per_Instruction: 0.00-1
Dhrystones_Per_Second_Per_MHz: -1
DMIPS_Per_MHz: 0.000
simulation finished @3137825ns

```

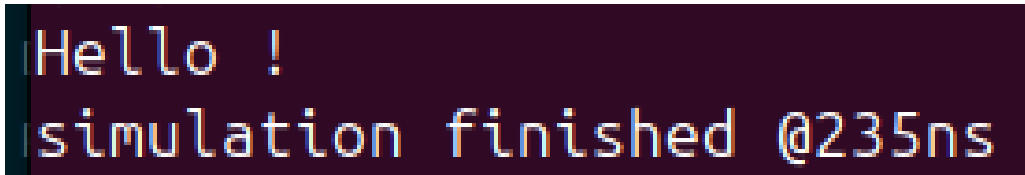
FIGURE 8 – Résultat du test 7-dhrystone – Partie 2

2.2 RISC_V_5STG pipeliné

Notre cœur RISC_V pipeliné fonctionne en simulation seulement sur les premiers tests : 1-hello_putc et 2-hello_putchar.

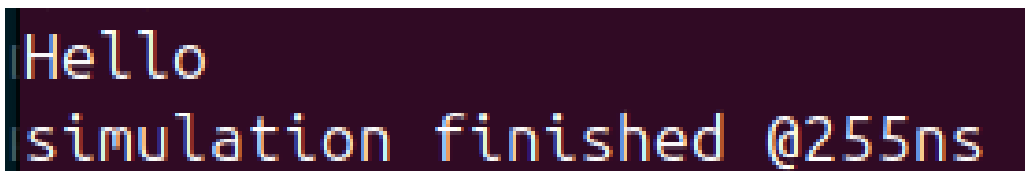
Il échoue sur les tests 3-hello_print et 3-vector-add. Et ne termine pas comme il faut sur les tests suivants.

Toutefois, on observe tout de même une augmentation des performances de 372% et 382% pour les deux premiers tests ce qui est très encourageant pour la suite.



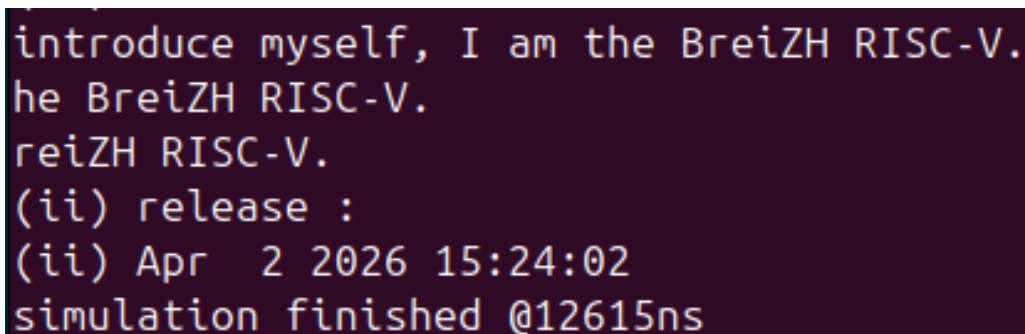
```
Hello !
simulation finished @235ns
```

FIGURE 9 – Résultat du test 1-hello_putc



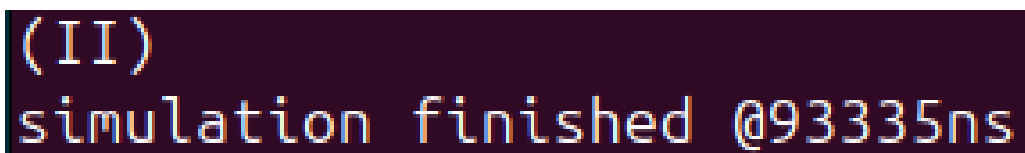
```
Hello
simulation finished @255ns
```

FIGURE 10 – Résultat du test 2-hello_putchar



```
introduce myself, I am the BreiZH RISC-V.
he BreiZH RISC-V.
reiZH RISC-V.
(ii) release :
(ii) Apr  2 2026 15:24:02
simulation finished @12615ns
```

FIGURE 11 – Résultat invalide du test 3-hello_print



```
(II)
simulation finished @93335ns
```

FIGURE 12 – Résultat invalide du test 3-vector-add